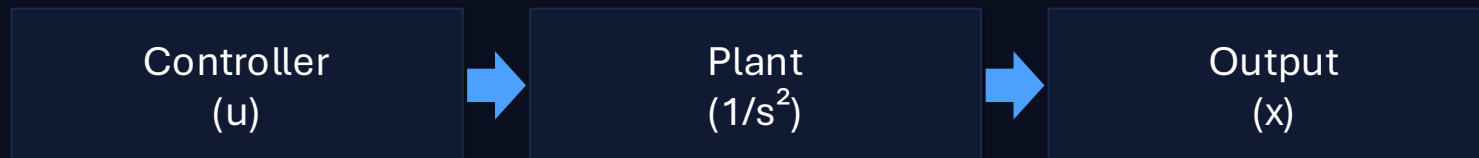




Double Integrator System

Theory + trajectory tracking control (PD + feedforward)



1) The double integrator model

Canonical second-order plant used for control design

Continuous-time dynamics (position/velocity)

$$\dot{x} = v$$

$$\dot{v} = u$$

$$\text{state: } X = [x \ v]^T$$

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$y = x$$

Key intuition

- u is the acceleration input (second derivative of position).
- Transfer function: $X(s)/U(s) = 1/s^2$ (two poles at the origin).
- Open-loop is marginal (does not return to zero by itself).

Why this model matters

- It is the local model of many mechanical systems (mass position) after feedback linearization.
- Used as a benchmark for tracking, optimal control, and MPC.
- Simple enough to derive stability and tuning rules by hand.

Controllability check

$$C = [B \ AB] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$
$$\text{rank}(C) = 2 \Rightarrow \text{controllable}$$

2) Regulation (go to the origin)

Full-state feedback for stabilization

State feedback

$$u = -K X = -k_p x - k_v v$$

Closed-loop:

$$\dot{x} = v$$

$$\dot{v} = -k_p x - k_v v$$

$$\Rightarrow \ddot{x} + k_v \dot{x} + k_p x = 0$$

If $k_p > 0$ and $k_v > 0 \Rightarrow$ asymptotically stable (2nd-order damped system).

Pole placement view

- Choose desired closed-loop poles p_1, p_2 (real and < 0).
- Characteristic polynomial: $(s - p_1)(s - p_2) = s^2 - (p_1 + p_2)s + p_1 p_2$.
- Match with $s^2 + k_v s + k_p \Rightarrow k_v = -(p_1 + p_2), k_p = p_1 p_2$.

Interpretation

- k_p sets “stiffness” (how strongly position error is corrected).
- k_v adds damping (reduces overshoot, shapes settling time).
- Same structure as a PD controller on position.

3) Trajectory tracking

Make $x(t)$ follow a time-varying reference $x_d(t)$

Goal

- Given a desired trajectory $x_d(t)$ (and usually $v_d(t)$, $a_d(t)$),
- Design $u(t)$ so that tracking error $e(t)=x(t)-x_d(t) \rightarrow 0$.
- Preferably with predictable transient response and robustness.

Define the error states

$$\begin{aligned} e &= x - x_d \\ \dot{e} &= v - v_d \\ \ddot{e} &= u - a_d \end{aligned}$$

Structure: feedforward + feedback



Total control: $u = a_d + u_{fb}$

- Feedforward cancels the desired acceleration.
- Feedback stabilizes the error dynamics.
- Together: accurate tracking with tunable transients.

4) PD + feedforward tracking control

Turn tracking into a stable 2nd-order error system

Control law

$$u(t) = a_d(t) - k_p (x - x_d) - k_v (v - v_d)$$

where:

x_d , v_d , a_d are desired
position/velocity/acceleration
Resulting error dynamics

$$\begin{aligned}\ddot{e} &= u - a_d = -k_v \dot{e} - k_p e \\ \Rightarrow \ddot{e} + k_v \dot{e} + k_p e &= 0\end{aligned}$$

If $k_p > 0$ and $k_v > 0$, the tracking error converges to 0 (for smooth x_d).

Tuning with (ω_n, ζ)

- Match: $\ddot{e} + 2\zeta\omega_n \dot{e} + \omega_n^2 e = 0$
- Choose ω_n for speed (rise time), ζ for damping (overshoot).
- Then: $k_v = 2\zeta\omega_n$, $k_p = \omega_n^2$.

Practical notes

- You need v (or an estimate) for the D term.
- Saturations turn the error dynamics nonlinear.
- For noisy velocity, consider filtering or an observer.

5) Choosing the reference trajectory

You typically need $x_d(t)$, $v_d(t)$, and $a_d(t)$

Example: sinusoidal tracking

$$\begin{aligned}x_d(t) &= A \sin(\omega t) \\v_d(t) &= A \omega \cos(\omega t) \\a_d(t) &= -A \omega^2 \sin(\omega t)\end{aligned}$$

- Smooth trajectories avoid large spikes in a_d (and therefore u).
- For point-to-point motion, use polynomials or splines (ensure continuous derivatives).
- Tracking gets harder as ω increases (you ask for higher accelerations).

Implementation checklist

- Define a function that returns (x_d, v_d, a_d) at time t .
- Compute errors e and $\dot{e}$ from the current state.
- Compute u , apply limits if needed, then integrate.
- Plot: x vs x_d , v vs v_d , and $u(t)$.

6) Practical considerations

Numerics, limits, and measurement quality

Saturation & anti-windup (if you add integral)

- Real actuators have $|u| \leq u_{\max}$ (acceleration/force bounds).
- With saturation, the error dynamics are no longer exactly linear.
- If steady-state bias exists, add integral action with anti-windup.

Sampling / discretization (real-time control)

- Controller is often implemented at a fixed rate (Δt).
- Keep ω_n well below the Nyquist limit ($\approx \pi/\Delta t$).
- Derivative on noisy signals amplifies noise $\rightarrow$ filter or estimate v .

Extensions

- LQR: $u = a_d - K [e, \dot{e}]^T$ (automatic trade-off between error and effort).
- MPC: handles input/state constraints explicitly.
- Observer/Kalman filter if only position is measured.

Rule of thumb

Start with $\zeta \approx 0.7$ and increase ω_n until you reach actuator limits or noise problems.

Summary

What to remember

Takeaways

- Double integrator: $\dot{x}=v$, $\dot{v}=u$ (transfer function $1/s^2$).
- Regulation: $u = -k_p x - k_v v \Rightarrow$ stable if $k_p, k_v > 0$.
- Tracking: $u = a_d - k_p(x-x_d) - k_v(v-v_d) \Rightarrow \ddot{e} + k_v \dot{e} + k_p e = 0$.
- Tune gains via poles or (ω_n, ζ) .
- Plot both states and control to see performance vs effort.

Next step: implement the controller in simulation (Python) and stress-test with noise + saturation.